MINI-PROJECT · DAY 3

Build Your First Bedrock Application

Zero to working — end-to-end. KB grounded. Guardrailed. Cost-tracked.

Time	4-8 hours (3 tiers — Core, Stretch, Advanced)
Format	Individual build
Tooling	Python + boto3 + Bedrock console. No Strands (you'll learn that later).
Deliverable	Working code · reflection write-up · cost analysis

WHAT YOU'RE BUILDING

A small but real Bedrock application. You pick the use case from three templates. You write the code yourself. You manage the conversation loop, the knowledge base, the guardrail, and the cost — every piece, by hand. By the end you'll have a working agent and a clear, intuitive grasp of what every Bedrock call actually does.

WHY NO FRAMEWORK YET

You will not use Strands or any agent framework. You're building the orchestration loop by hand on purpose. When you learn Strands later, you will know — line for line — what it's collapsing for you. That clarity is the whole point of this project.

PART 1 · HOW THIS WORKS

Three tiers in one project

Everyone ships the Core tier. Stretch and Advanced are for when you're done early or want to push further. Time estimates are realistic for a learner who's followed Day 1-3.

Tier	Time	What you ship
Core	**4 hours**	Single-turn, KB-grounded Q&A bot with one Guardrail. Works end-to-end.
Stretch	**+2 hours**	Multi-turn conversation + one custom tool. Now it's an assistant.
Advanced	**+2 hours**	Cost dashboard + simple evaluation harness + multi-tool routing.

> **GRADING PRINCIPLE**
> Core complete and working = a full pass. Stretch and Advanced raise the ceiling but do not rescue a broken Core. **Ship Core first. Then climb.**

PART 2 · CONCEPTS YOU'VE LEARNED

Things this project pulls from every session so far

If something below feels unfamiliar, go back to the relevant deck before starting. The project assumes all of this is in working memory.

Day 1	**LLM foundations**	Tokenization, why the model is stateless, how training data has a cutoff.
Day 2	**Bedrock landscape**	What Bedrock is, available models, IAM/region setup.
Day 3 · Ch 1-2	**Converse API**	boto3 client, the `us.` model prefix, system prompt vs user message.
Day 3 · Ch 3-4	**Tokens + multi-turn**	Token economics, manual `conversation_history` management, context window.
Day 3 · Ch 5	**Tool use**	The 8-step loop, tool schemas, dispatch tables, two-call pattern, `toolUseId`.
Day 3 · Ch 6	**Knowledge Bases**	S3 ingestion, embeddings, `retrieve_and_generate`, citations.
Day 3 · Ch 7	**Guardrails**	Five filter types, console-only setup, versioning, applying to Converse and KB calls.

PART 3 · PICK A USE CASE

Three templates — pick one

Each template gives you a real-world problem. The shape of the work is identical across all three. Pick the one whose data and domain you find most interesting to work with — you'll be staring at it for several hours.

TEMPLATE A · Internal Knowledge Assistant

Employees ask HR/IT policy questions and get grounded answers from internal docs.

Knowledge base contents	10-15 short markdown docs you'll write: vacation policy, expense policy, laptop request process, onboarding FAQ, security guidelines, etc.
Tools (Stretch tier)	`get_vacation_balance(employee_id)` · `open_it_ticket(category, description)`
Good fit if…	If you like writing and want practice with concise policy documents.

TEMPLATE B · Customer Support Bot

Customers ask product questions, get answers from manuals/FAQs, and route complex issues to humans.

Knowledge base contents	10-15 docs covering a fictional product line — installation guide, common errors, return policy, warranty, shipping, troubleshooting steps.
Tools (Stretch tier)	`check_order_status(order_id)` · `create_support_ticket(issue, priority)`
Good fit if…	If you've worked in customer ops or want to think about escalation flows.

TEMPLATE C · Research / Analysis Assistant

A researcher uploads source documents and asks comparative/summary questions across them.

Knowledge base contents	10-15 short summaries of papers, articles, or product reviews in a domain you pick (could be tech trends, financial reports, scientific abstracts).
Tools (Stretch tier)	`get_current_date()` · `fetch_market_data(ticker)` (mock)
Good fit if…	If you want to practice prompt engineering for analysis and comparison.

PART 4 · CORE TIER (4 HOURS)

What ships in Core

Every item below is required. Do these in order — each step builds on the previous one. When all twelve are checked, your Core build is done.

1. Set up your environment

Install boto3. Configure AWS credentials. Confirm you can list Bedrock models in your region. Pick your AWS region and stick with it for the whole project.

2. Pick a use case

Choose Template A, B, or C. Write a one-paragraph problem statement: who the user is, what they ask, what success looks like. This is your BRD.

3. Write your knowledge base content

Create 10-15 short markdown documents in `/kb_docs`. Real-feeling content — dates, dollar amounts, policy specifics. Each doc 200-500 words. Variety in style and length is good.

4. Upload to S3 and create the Knowledge Base

Upload your docs to an S3 bucket. In the Bedrock console, create a Knowledge Base pointing at the bucket. Use Amazon Titan Embed as the embedding model. Use S3 Vectors as the vector store. Sync. Note the KB ID.

5. Verify retrieval in the console

Before writing any code, use the console's test interface to query the KB with 3-4 sample questions. Confirm it returns relevant chunks. **This catches half of all KB issues before they hit your code.**

6. Create a Guardrail

In the Bedrock console, create a Guardrail. Enable: content filters (medium severity), and one denied topic relevant to your use case. Create version 1. Note the Guardrail ID.

7. Write a first-call script

Write `scripts/01_hello_bedrock.py` — a single Converse call to your chosen model with a simple system prompt. Confirm you can call Bedrock at all. Print the token usage.

8. Add KB grounding

Write `scripts/02_kb_query.py` using `retrieve_and_generate`. Ask one of the questions you tested in step 5. Print the answer AND the citations. If citations are empty, something's wrong — fix before continuing.

9. Attach the Guardrail

Add the guardrail config to your KB call. Run the same query — confirm it still works. Then deliberately send a denied query — confirm the guardrail blocks it.

10. Build the main app

Combine into `scripts/03_app.py`: read a question from input(), call KB+Guardrail, print the answer with citations, print token usage. Loop until the user types 'quit'. This is your minimum viable Bedrock app.

11. Track cost

Open the Cost Analysis worksheet (separate file). Run 10 queries through your app. Log input/output tokens for each. Compute total cost per session. Estimate cost per 1,000 users/day.

12. Write the reflection

Open the Reflection Template. Answer all six prompts. Honest answers — not polished ones. What broke? What surprised you? What would you do differently?

> **DEFINITION OF DONE — CORE**
> Your app reads a question, returns a KB-grounded answer with citations, blocks denied queries via the guardrail, and prints token usage. The cost worksheet has 10 real entries. The reflection has six answered prompts. Ship that → Core is done.

PART 5 · STRETCH TIER (+2 HOURS)

Add multi-turn and one custom tool

Now your app stops being a Q&A; box and starts behaving like an assistant. These two additions are where the manual orchestration work pays off.

S1. Multi-turn conversation

Replace the single-turn loop in `03_app.py` with a `conversation_history` list that grows turn over turn. Append both user and assistant messages each turn. Pass the full history on every call. Test that the bot remembers context from earlier turns.

S2. Pick and write one custom tool

Choose one tool from your template's Stretch list (or invent one). Implement it as a Python function in `scripts/tools.py`. Mock the data — no real backend needed. Return realistic JSON.

S3. Wire the tool into the Converse loop

Write the 8-step tool-use flow by hand. Add the `toolConfig` to your Converse calls. Handle `stop_reason == "tool_use"`. Append both the assistant tool-use message AND the tool result. Second Converse call returns the final answer.

S4. Decide: KB or Tool per query

Your app now has two ways to answer: retrieve from KB, or call a tool. Let the model decide via the system prompt and tool schema. Test with 4-5 different questions — some should hit the KB, some should hit the tool, some both.

S5. Re-run cost tracking

Update the cost worksheet. Tool use means MORE token cost — the model sees the tool schema, the tool result, and writes a follow-up answer. Quantify the difference.

> **DEFINITION OF DONE — STRETCH**
> Multi-turn conversation works. One custom tool is callable from within the conversation. Cost worksheet shows the before/after difference of adding tool use. Your reflection now includes a section on tool design choices.

PART 6 · ADVANCED TIER (+2 HOURS)

Pick one (or more) of these challenges

These deliberately push past what was demoed. Pick what aligns with where you want to grow. No starter snippets — figure it out from the AWS docs and your existing code.

A1. Cost dashboard

Build a `scripts/04_dashboard.py` that reads your cost log and prints a small report: average tokens per turn, cost per turn, projected monthly cost at 100 / 1,000 / 10,000 users/day. Bonus: write it as a Markdown table.

A2. Evaluation harness

Write a list of 10 test questions with expected behaviors (which KB doc should be cited, whether a tool should fire, whether the guardrail should block). Run all 10 through your app and score yourself. This is the simplest possible eval — production teams build this on day one.

A3. Multi-tool routing (no framework)

Add a second tool to your app. Two tools means the model has to pick — make the tool descriptions different enough that it always picks the right one. Test with adversarial inputs.

A4. Prompt caching

If your model supports it, enable prompt caching for the stable system-prompt prefix. Measure the cost difference across a 10-turn conversation. (Hint: research the Bedrock `cachePoint` API.)

A5. Streaming responses

Replace `converse` with `converse_stream`. Print tokens as they arrive instead of waiting for the full response. Reflection: how does this change the UX? Does it change the cost?

A6. Persist conversations

Save each conversation to a local JSON file. Next time the user returns, load it and continue. Watch how the input-tokens-per-turn bar climbs when you load a long history.

PART 7 · SUGGESTED FILE STRUCTURE

Where to put what

You can deviate from this. But if you don't have a strong preference, follow it. Names matter — when you debug at midnight, you'll thank yourself.

```
bedrock-miniproject/
├── README.md                  # one-page project summary + how to run
├── .env.example               # AWS_REGION, KB_ID, GUARDRAIL_ID, GUARDRAIL_VERSION
├── .gitignore                 # ignore .env, __pycache__, cost_log.json
├── kb_docs/                   # 10-15 markdown docs that you wrote
│   ├── doc_01.md
│   ├── doc_02.md
│   └── …
├── scripts/
│   ├── 01_hello_bedrock.py    # Step 7: first Converse call
│   ├── 02_kb_query.py         # Step 8-9: retrieve_and_generate + guardrail
│   ├── 03_app.py              # Step 10: the main app
│   ├── tools.py               # Stretch: tool definitions
│   └── 04_dashboard.py        # Advanced A1: cost dashboard
├── cost_log.json              # appended to by 03_app.py on every turn
├── reflection.md              # the reflection write-up (provided template)
└── cost_analysis.xlsx         # the cost worksheet (provided)
```

> **ON THE .ENV FILE**
> Never commit your AWS credentials or your Knowledge Base ID. Use a .env file locally (read it with `python-dotenv` or `os.environ`). The .env.example file ships in your repo with the variable names but no values.

PART 8 · STARTER SNIPPETS

Three snippets to get unstuck

Use these as a launching point — not as copy-paste. Typing them yourself reinforces what each line does. Full versions ship as separate files in your starter pack.

Snippet 1 — Hello Bedrock (Step 7)

```python
import os, boto3

REGION = os.getenv("AWS_REGION", "us-east-1")
MODEL_ID = "us.amazon.nova-lite-v1:0"

client = boto3.client("bedrock-runtime", region_name=REGION)

resp = client.converse(
    modelId=MODEL_ID,
    system=[{"text": "You are a helpful assistant. Be concise."}],
    messages=[{"role": "user", "content": [{"text": "What is Amazon Bedrock?"}]}],
    inferenceConfig={"temperature": 0.3, "maxTokens": 300},
)
print(resp["output"]["message"]["content"][0]["text"])
print("Usage:", resp.get("usage"))
```

Snippet 2 — KB query with Guardrail (Steps 8-9)

```
import os, boto3

REGION = os.getenv("AWS_REGION", "us-east-1")
KB_ID = os.environ["KB_ID"]
GUARDRAIL_ID = os.environ["GUARDRAIL_ID"]
GUARDRAIL_VERSION = os.environ["GUARDRAIL_VERSION"]
MODEL_ARN = "us.amazon.nova-lite-v1:0"

agent = boto3.client("bedrock-agent-runtime", region_name=REGION)

resp = agent.retrieve_and_generate(
    input={"text": "What's our remote-work policy?"},
    retrieveAndGenerateConfiguration={
        "type": "KNOWLEDGE_BASE",
        "knowledgeBaseConfiguration": {
            "knowledgeBaseId": KB_ID,
            "modelArn": MODEL_ARN,
            "generationConfiguration": {
                "guardrailConfiguration": {
                    "guardrailId": GUARDRAIL_ID,
                    "guardrailVersion": GUARDRAIL_VERSION,
                },
            },
        },
    },
)
print(resp["output"]["text"])
for c in resp.get("citations", []):
    for ref in c.get("retrievedReferences", []):
        print("  cite:", ref.get("location", {}).get("s3Location", {}).get("uri"))
```

Snippet 3 — Multi-turn loop (Stretch S1)

```
conversation_history = []

def ask(user_msg):
    conversation_history.append({
        "role": "user",
        "content": [{"text": user_msg}],
    })
    resp = client.converse(
        modelId=MODEL_ID,
        system=[{"text": SYSTEM_PROMPT}],
        messages=conversation_history,
        inferenceConfig={"temperature": 0.3, "maxTokens": 600},
    )
    assistant_msg = resp["output"]["message"]
    conversation_history.append(assistant_msg)        # <-- critical
    return assistant_msg["content"][0]["text"], resp.get("usage", {})
```

THE MOST COMMON BUG

Forgetting to append the assistant's response to `conversation_history`. If you only append the user message, the bot has no memory. If you also forget to pass `conversation_history` as `messages`, you're not multi-turn at all. Both lines must exist together.

PART 9 · DELIVERABLES

What you turn in

1. The code repo

A folder following the suggested structure. `README.md` explains what you built and how to run it. `.env.example` committed; `.env` not committed.

2. reflection.md

Six prompts answered (template provided). Honest, specific, not polished. Errors and surprises are the most valuable parts.

3. cost_analysis.xlsx

10 real queries logged. Per-query tokens. Projected monthly cost at 3 user-volume tiers. Worksheet provided.

4. (Stretch+) Updated reflection

Add a section on what changed between Core and Stretch — tool design, cost impact, what was hard about multi-turn.

5. (Advanced+) One advanced challenge picked + completed

Document which challenge, how you approached it, and the result.

Grading rubric

Area	Weight	What earns full marks
Code works end-to-end (Core)	**30%**	App runs, reads a question, returns a KB-grounded answer with citations, guardrail blocks denied queries.
Code quality + structure	**15%**	Files in the suggested places. Env vars used. No hardcoded secrets. Reads cleanly.
Cost analysis	**15%**	10 real queries logged. Per-query tokens captured. Monthly projections at 3 volume tiers.
Reflection write-up	**20%**	All six prompts answered specifically. Errors and surprises are described in detail.
Stretch tier (multi-turn + 1 tool)	**+15%**	Multi-turn works, one tool wires through the 8-step loop, cost impact documented.
Advanced tier (any one challenge)	**+5%**	One advanced challenge documented and working.

PART 10 · TIME BUDGET

A realistic plan for 4-8 hours

Hour	What you're doing	Tier
0:00 – 0:30	Read this brief. Pick a template. Write your one-paragraph BRD.	Core
0:30 – 1:30	Write 10-15 KB documents. Upload to S3. Create the KB. Wait for sync.	Core
1:30 – 2:00	Test retrieval in the console. Create the guardrail.	Core
2:00 – 3:00	Write 01_hello_bedrock.py and 02_kb_query.py. Confirm both work.	Core
3:00 – 3:45	Build 03_app.py — the main loop. Single-turn. Working end-to-end.	Core
3:45 – 4:00	Run 10 queries. Fill the cost worksheet. Start the reflection.	Core
— 4 hour mark —		
4:00 – 5:00	Multi-turn — make 03_app.py stateful. Test memory across turns.	Stretch
5:00 – 6:00	Write one tool. Wire the 8-step loop. Re-log costs.	Stretch
— 6 hour mark —		
6:00 – 8:00	Pick one Advanced challenge. Implement. Document.	Advanced

A few practical notes before you start

- **Bedrock model availability varies by region.** If Nova or Claude isn't available in your region, pick the closest equivalent. Note your choice in the reflection.
- **KB sync takes time.** Start the sync as soon as you have docs uploaded. Do other work while it runs.
- **Guardrails block fast.** When testing, deliberately send a query that should be blocked. If it isn't, your guardrail isn't actually attached. Common mistake.
- **Print token usage everywhere.** Make every script print `resp['usage']`. You can't optimize what you can't see.
- **Citations are not optional.** A KB answer without citations means RAG isn't working. Don't accept it.
- **The reflection is graded.** Errors, surprises, and "what I'd do differently" are worth as much as the code. Be specific.

> **ONE LAST THING**
> When you finish, you'll have built — by hand — the same orchestration that Strands collapses into three lines. When that lesson comes, you'll feel the payoff. That's the point.